

Using Vision Trasformers in Federated Learning

Leonardo Rolandi
Politecnico di Torino
s301216

Giuseppe Pellegrino
Politecnico di Torino
s303999

Flavio Patti
Politecnico di Torino
s301104

Abstract—Federated learning is a decentralized machine learning approach that enables the training of models on data distributed across multiple devices without the need for centralized data aggregation. This approach offers various advantages such as preserving data privacy, reducing communication costs, and enabling collaborative learning. However, federated learning also poses various challenges such as statistical heterogeneity, system heterogeneity, and privacy and security concerns, which require specialized techniques and solutions. For these reasons, in this paper, we focus our attention on FedAvg for statistical heterogeneity, FedMD for system heterogeneity and evaluating possible privacy breaches using gradient inversion attack; comparing the performances obtained by ResNet20 both on the federative and the centralized environments. In particular, focusing only on FL, we compare the performances reached by ResNet20 between different kinds of architectures (such as ConvNet and Vision Trasformers), in order to better understand how much the choice of the model affects these algorithms both in terms of timing and results

Index Terms—Federated learning, FedAvg, FedMD, ResNet20, Vision Trasformer, Statistical Heterogeneity, System Heterogeneity, Gradient Inversion Attack, Privacy

I. INTRODUCTION

Nowadays, a lot of data cannot be exploited by traditional deep learning methods due to the sensitive and protected nature of privacy. Federated Learning is a branch of Machine Learning scenario that aims to use privacy-protected data without violating current legislations. The goal is to learn a global model in scenarios with privacy constraints by leveraging a client-server architecture. The central model is maintained on the server side and, unlike the standard deep learning setup, does not have direct access to the data, which are stored on multiple devices, i.e. the clients. Thanks to a paradigm based on the exchange of model parameters between client and server through multiple rounds, the global model is able to extract knowledge from data without violating user privacy. Although the Federated Learning approach is capable of providing a way to train neural networks efficiently, there are three main challenges on which we have to face with:

- *Statistical Heterogeneity* i.e. the non-IID (independent and identically distributed) client data which leads to degraded and unstable learning.
- *System Heterogeneity* where participants have devices with different computational capabilities.
- *Security and privacy* issues deriving from the possibility of obtain informations on client data from the gradients of the model exchanged on the network, even if it's much harder than the traditional distributed algorithms

When Federated Learning took hold, model updates were thought to contain almost no informations on the original data, and they were considered impossible to be leaked. In reality it has been shown that the gradients of the parameters still contain significant informations about private input data, in particular it has been shown that even on a batch of 100 images, many of these are still recoverable.

In this paper, we analyze several architectures for different FL scenarios, including the ones related to the challenges previously mentioned. In particular, we focus on the comparison between the performances of ResNet20 [9] and Vision Trasformers (ViTs) [14] architectures, trying to analyze why it may be useful to choose the latter in the FL scenario, what are the pros and cons of using this kind of architecture and how the computational complexity varies. For the experiments, we considered the CIFAR100 [8] dataset. The analysis was done by tracking different measurements such as accuracy and loss.

II. RELATED WORK

Federated learning algorithms such as FedAvg [1] enable the decentralized training of a model, where clients acquire the updated model parameters from the server and enhance them through multiple gradient descent steps using their own private data. However, in the presence of non-identical client data distributions, this approach produces low performances, characterized by unstable learning and slower convergence. Numerous researchers have investigated the FedAvg algorithm using synthetic non-identical client data partitions created from image classification datasets. McMahan et al. [2] has generated pathological non-identical user splits by manipulating the MNIST dataset, sorting training examples based on class labels and dividing them into shards so that each client received 2 shards. They demonstrate that FedAvg on non-identical clients still converges to 99% accuracy, though taking more rounds than identically distributed clients. Other authors look at more realistic data distributions at the client. For example, Caldas et al. [3] use Extended MNIST [4] with partitions over writers of the digits, rather than simply partitioning over digit class, and there the convergence is more difficult.

Federated Learning encounters various challenges, and one of them is addressed by the FedMD algorithm [5]. FedMD tackles the issue of system heterogeneity among participating agents by enabling the utilization of different network architectures on each client. This is achieved through the utilization of transfer learning and knowledge distillation [6], which facilitate communication and translation of model updates

between all participants using class scores from a public dataset.

Additionally, privacy and security concerns are prominent in the context of Federated Learning. The main problem in this context, in effect, is the privacy of users' data, which has been shown to be at risk due to the Gradient Inversion Attack [7]. This attack exploits the information contained in gradient values communicated to the server, allowing for the recovery of private user information.

III. METHOD

A. Vision Transformer (ViT)

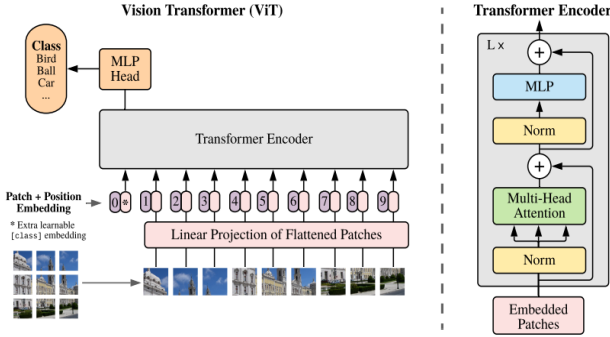


Fig. 1. Model overview. We split an image into fixed-size patches, linearly embed each of them, add position embeddings, and feed the resulting sequence of vectors to a standard Transformer encoder. In order to perform classification, we use the standard approach of adding an extra learnable “classification token” to the sequence.

An overview of the Vision Transformer model is depicted in Figure 1. The standard Transformer receives as input a 1D sequence of token embeddings. To handle 2D images, as written in [14], we reshape the image $\mathbf{x} \in \mathbb{R}^{H \times W \times C}$ into a sequence of flattened 2D patches $\mathbf{x}_p \in \mathbb{R}^{N \times (P^2 \cdot C)}$, where (H, W) is the resolution of the original image, C is the number of channels, (P, P) is the resolution of each image patch, and $N = H \times W / P^2$ is the resulting number of patches, which also serves as the effective input sequence length for the Transformer. The Transformer uses constant latent vector size D through all of its layers, so we flatten the patches and map to D dimensions with a trainable linear projection (1), where $\mathbf{E} \in \mathbb{R}^{(P^2 \cdot C) \times D}$ and $\mathbf{E}_{pos} \in \mathbb{R}^{(N+1) \times D}$. We refer to the output of this projection as the patch embeddings.

We prepend a learnable embedding to the sequence of embedded patches ($\mathbf{z}_0^0 = \mathbf{x}_{class}$), whose state, at the output of the Transformer encoder, ($\mathbf{z}_L^0$) is used as the image representation $\mathbf{y}$ (4). A classification head is attached to $\mathbf{z}_L^0$ and it is implemented by a single linear layer.

Position embeddings are added to the patch embeddings to retain positional information. We use standard learnable 1D position embeddings. The resulting sequence of embedding vectors serves as input to the encoder.

The Transformer encoder consists of alternating layers of

multiheaded self-attention (MSA, see Appendix A) and MLP blocks (2), (3). Layernorm (LN) is applied before every block, and residual connections after every block.

The MLP contains two layers with a GELU non-linearity.

$$\mathbf{z}_0 = [x_{class}; \mathbf{x}_p^1 \mathbf{E}; \mathbf{x}_p^2 \mathbf{E}; \dots; \mathbf{x}_p^N \mathbf{E}] + \mathbf{E}_{pos} \quad (1)$$

$$\mathbf{z}_\ell' = MSA(LN(\mathbf{z}_{\ell-1})) + \mathbf{z}_{\ell-1}, \quad \ell = 1 \dots L \quad (2)$$

$$\mathbf{z}_\ell' = MLP(LN(\mathbf{z}_\ell')) + \mathbf{z}_\ell', \quad \ell = 1 \dots L \quad (3)$$

$$\mathbf{y} = LN(\mathbf{z}_L^0) \quad (4)$$

B. ResNet20

The main architecture used in the experiments is ResNet20. It's a residual convolutional network with 20 layers that has been carefully designed for image processing tasks. It operates on input images that have dimensions of $3 \times 32 \times 32$. The scheme of the network can be seen in Figure VI-B and consists of 3 main blocks, each of them are made up of 6 convolution layers (except for the first one which contains 7 convolution layers) that progressively double the number of channels and halve the shape of the original input using a stride $s=2$.

Towards the end of the network, an Average Pooling is performed using a kernel size of 8×8 . This pooling process condenses the values and creates a summary representation. These summarized values are then flattened and passed to a fully connected layer consisting of m neurons, where m corresponds to the number of distinct classes to be predicted.

C. Definition of a Centralized Baseline

To establish a baseline for our experiments, the ResNet20 model was trained until convergence in a centralized setting, employing the Cross Entropy function as loss and Stochastic Gradient Descent (SGD) as optimizer. The model's performance was evaluated using the accuracy metric. The obtained results serve as an upper bound for the experiments conducted in a federated environment. Several experiments were conducted to examine the impact of various normalization techniques and learning rate schedulers.

D. Statistical heterogeneity

Since statistical heterogeneity is one of the biggest challenges FL has to overcome, we analyzed FedAvg [1] that defines the basis of most of the federated learning algorithms, where convergence of a centralized model (the server) is based on a weighted average of the local models (also called clients). For every federated round, K clients are randomly selected with uniform probability from a pool of active clients. Selected clients, indexed by k , download the same starting model w_t from a central server and perform local SGD optimization, minimizing an empirical loss $L(b)$ over local mini-batches b with learning rate η , for E epochs before sending the model update w_{t+1}^k back to the server. The server then averages the updates from the clients with weights proportional to the sizes of clients' local data.

Algorithm 1 explains better what is described in this section through pseudo code.

Algorithm 1 FederatedAveraging. The K clients are indexed by k ; B is the local minibatch size, E is the number of local epochs, and η is the learning rate.

Server executes:

initialize w_0

for each round $t = 1, 2, \dots$ **do**

$S_t \leftarrow$ (random set of m clients)

for each client $k \in S_t$ **in parallel do**

$w_{t+1}^k \leftarrow \text{ClientUpdate}(k, w_t)$

end for

$w_{t+1} \leftarrow \sum_{k=1}^K \frac{n_k}{n} w_{t+1}^k$

end for

ClientUpdate(k, w):

for each local mini-batch b over E epochs **do**

$w \leftarrow w - \eta \nabla \ell(w; b)$

end for

return w to server

E. System heterogeneity

In this part we try to test the mentioned architectures in a system heterogeneity context, in order to understand how they would behave in the most realistic possible situation. In effect, customers, in a business facing setting, create their model according to their needs, so a situation where each participant has a different model with different bandwidth and computational capacity is very common. FedMD [5] is one of the possible solutions to perform federated learning when each participant has a different model, unknown to the others. Like any framework for heterogeneous FL, there are the agent, the translator and the consensus, where information is shared among the models. Each participant k has a small private dataset D_k and in addition has the ability to access a larger public dataset D_0 that will serve as the basis for communication with the other models. At the beginning, each model f_k is trained first on the public dataset and then on the private dataset, and this will provide a baseline that is the starting point for each model. That done, the collaboration phase begin. To avoid communication burden, each party computes class scores on a subset of the public dataset d_j and sends them to a central server, where the average of each participant's scores will be computed and this represents the updated consensus. Each f_k model will then be trained to approach the consensus on d_j and finally be trained on its private dataset D_k for a few epochs again. Through the use of techniques such as transfer learning and knowledge distillation, we are able to apply federate learning in a context of system heterogeneity without sharing personal client data or data about the models used.

F. Privacy

The idea behind federated learning is to train a neural network in a collaborative manner with a common a server to whom each user share only the gradients instead of the original data, and this is because the goal is not only to train a model

efficiently, but also to shield the input data and ensure privacy among the various collaborating users. But this work about gradient inversion attack [7] shows us that sharing gradients is not so safe, and in fact, it is possible to reconstruct input data (in this case, images) from the knowledge of gradients, even for deep networks that have already been trained. It is indeed shown that gradients can carry great information about the input data. In order to reconstruct an image, we need a cost function and an optimizer. Previous algorithms have used a Euclidean cost function:

$$\arg \min_x \|\Delta_\theta \mathcal{L}_\theta(x, y) - \Delta_\theta \mathcal{L}_\theta(x^*, y)\|^2 \quad (5)$$

In this case x is minimized in order to recover the original image input x^* from the transmitted gradient $\Delta_\theta \mathcal{L}_\theta(x^*, y)$, and this problem is solved by an L-BFGS solver [16]. This type of solution works more on untrained or at least shallow networks, so it is not good for a real-world scenario. The proposed solution decomposes the parameter gradient into magnitude and direction, and it shows that the direction of the gradient can bring significant informations. For this reason a cost function that is based on angles will be used (i.e. cosine similarity) and in this case the objective is not to find images with a gradient that fit well with observed gradients, but to find images that lead to a similar change in model prediction as the ground truth.

$$\arg \min_{x \in [0,1]^n} 1 - \frac{\langle \nabla_\theta \mathcal{L}_\theta(x, y), \nabla_\theta \mathcal{L}_\theta(x^*, y) \rangle}{\|\nabla_\theta \mathcal{L}_\theta(x, y)\| \|\nabla_\theta \mathcal{L}_\theta(x^*, y)\|} + \alpha \text{TV}(x) \quad (6)$$

In particular, the total variation (TV) parameter measures the sum of the absolute differences between adjacent pixels in an image. Introducing total variation as a regularization in image inversion promotes the generation of smoother and more coherent images while reducing unwanted artifacts and noise.

This cost function is minimized only based on the sign of its gradient, and is optimized with Adam [15].

IV. EXPERIMENTS

The goal of this paper is to analyze the performances of existing algorithms for different scenarios of FL, in particular, we focus on FedAvg for statistical heterogeneity and FedMD for system heterogeneity. So several experiments were carried out, using different hyperparameters and architectures; comparing them to a centralized baseline (used as upper bound) in order to highlight their strengths and weaknesses. For the centralized baseline we use CIFAR100 [8] as dataset on a ResNet20 [9] network using the following setup:

- **Learning Rate** = 0.1
- **Weight Decay** = 10e-4
- **Momentum** = 0.9
- **Batch size** = 128
- **Epochs** = 160

In addition, our analyses were done using Group Normalization (GN) layers [10] and Batch Normalization (BN) ones and for each of them we tested also two different kind of learning rate scheduler: the classical Step LR with a decay

factor $\gamma = 0.1$ and a step size of 30 epochs and the Cosine Annealing LR [12] with T_{max} value equal to the number of epochs. Results can be seen in Table I.

For the Group Normalization we finetune different values of $C = [2, 4, 8]$ and we have noticed that with $C = 2$ we have obtained best results, so Table I refers to this configuration of GN. The network weights were initialized using Kaiming weight initialization as in [11]. A standard data augmentation pipeline was applied, with a random crop, random horizontal flip with probability $p = 0.5$, and per-pixel normalization with mean and standard deviation of the training set of CIFAR100. Batch normalization layers in the centralized setting give us slightly better results than Group normalization, but as stated by [10] Group Normalization are usually more effective than Batch Normalization in the federated scenario.

TABLE I
CENTRALIZED RESNET20

Normalization	LR scheduler	Accuracy (%)	Parameters
Batch Normalization	Cosine Allealing	64.07	27,8M
Batch Normalization	Step	63.48	27,8M
Group Normalization	Cosine Allealing	61.78	27,8M
Group Normalization	Step	59	27,8M

A. Statistical heterogeneity

We conducted an analysis using the FedAvg algorithm on two types of scenarios: independent and identically distributed (IID) data, and non-IID data. To simulate these scenarios, we used the Dirichlet distribution as in [13] with $\alpha = [0, 0.5, 1000]$. In particular $\alpha = 1000$ represented the IID scenario, while $\alpha = [0, 0.5]$ represented the non-IID scenario. The dataset was equally divided among 100 clients. In each round, a randomly selected subset of clients participate, with a 10% participation rate and equal probability for selection. We test the performances of the previous mentioned ResNet20 with respect to three different configurations of Vision Trasformer (ViT):

- **Vision Trasformer Small (ViT_S):**

- patch_size = 4x4
- hidden_size = 384
- mlp_dim = 1536
- num_heads = 6
- num_layers = 12

- **Vision Trasformer Base (ViT_B):**

- patch_size = 4x4
- hidden_size = 768
- mlp_dim = 3072
- num_heads = 12
- num_layers = 12

- **Vision Trasformer Large (ViT_L):**

- patch_size = 8x8
- hidden_size = 1024
- mlp_dim = 4096
- num_heads = 16

– num_layers = 24

Each network was trained for 5000 rounds, with 1 epoch per round on each client. Furthermore, for ViTs we use a batch size of 512 while for ResNet20 it's fixed to 128; this choice is due to the fact that using attention modules, generally, ViTs work better by analyzing large numbers of data simultaneously. For the FedAvg algorithm, we employed a learning rate of $\eta_c = 0.1$ for the clients and $\eta_s = 1$ for the server. Additionally, a weight decay of 0 was chosen. As can be seen from Figure 2 and Table II, in IID conditions, ResNet20 turns out to be the best, reaching an accuracy of 54.86%, while the three types of ViTs are almost equivalent, reaching only an accuracy of 44.35%(ViT_S). Things change substantially in non-IID scenarios, in fact in this case, both ViT_S and ViT_B manage to exceed the accuracy of the ResNet by 13%, reaching a test accuracy of 25% after 5000 rounds. The ViT_L model, on the other hand, continues to respond poorly compared to the other models, this is probably due to the fact that it was trained with 8x8 patches size ¹ instead of 4x4, therefore working on images of size 32x32 and having only 16 patches available it is unable to learn as it should.

TABLE II
TEST ACCURACY (%) OVER 5000 ROUNDS FOR STATISTICAL HETEROGENEITY ANALYSIS

Model	IID ($\alpha = 1000$)	Non-IID ($\alpha = 0.5$)	Non-IID ($\alpha = 0$)
ResNet20	54.86	20.97	11.41
ViT_S	44.35	29.62	24.25
ViT_B	42.09	28.76	24.14
ViT_L	40.56	11.64	12.42

B. System heterogeneity

Regarding the system heterogeneity scenario we used 3 different types of models, for a total of 10 models:

- **5 Convolutional Neural Networks:** 3 with two convolutional layers and 2 with three convolutional layers, with different dimension and dropout rate.
- **2 ResNets:** One applying batch normalization and one with group normalization, the one used in the centralized training phase
- **3 Vision Transformers:** Which correspond to those used previously in the FedAvg phase: ViT_S, ViT_B, ViT_L

For the environment, we used CIFAR10 as the public dataset and subsets of CIFAR100 as private datasets. At the beginning, as mentioned previously, each participant trained their models on the public dataset, obtaining a range of accuracy from 55% up to 78%, then the networks were trained on their private dataset, the results obtained in this phase are the starting point for the next phase, the collaborative training. During this phase the upper bounds were also computed, by training the

¹The reason for this choice was that using only Colab Pro it was not possible to run the three types of ViTs, therefore we asked and obtained permission to run jobs on the Politecnico's cluster (HPC@POLITO) where, instead, it was possible to obtain these results using 4x4 patches on ViT_S and ViT_B but not for the ViT_L due to GPU memory limitations.

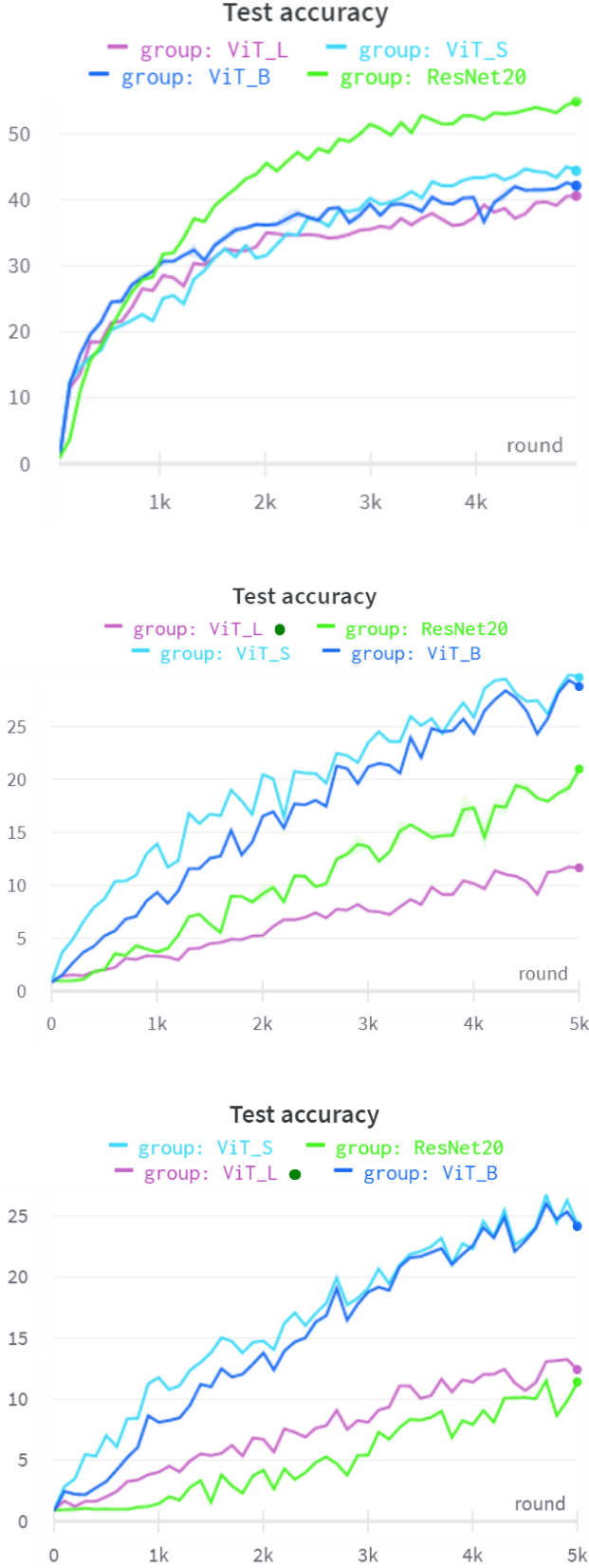


Fig. 2. FedAvg results with different values of α after 5000 rounds:
Top: $\alpha = 1000$, Center: $\alpha = 0.5$, Bottom: $\alpha = 0$

models on all private datasets joined. Once the collaborative training phase began, we can see from the graphs in the Figure 3 that the models improve instantaneously and then settles down around a certain level of accuracy, which is why we decided not to exceed 20 steps. We used two different types of optimizers: For CNNs and Vision Transformers we used the Adam optimizer [15] with an initial learning rate of 0.001, while for ResNets we used the SGD, with an initial learning rate of 0.1. Again, performances under conditions of non identical distribution were investigated: a subset of the superclasses contained in CIFAR100 was chosen, during the training phase at each participant was given samples belonging to only one subclass for each superclass, so each of them had different subclasses. In the testing phase, the goal was to classify generic data, not belonging to the same subclass used in the training phase, into the correct superclass. As can be seen from the graphs the models are struggling to reach their upper bounds and in the IID environment the performances are better. The summary can be found in table III. The best results are obtained by the ResNets, in particular the version with the batch normalization reaches 25 percentage points over the baseline, followed by the CNNs, which improve on average 12 percent in 20 rounds, and finally the Vision Transformers seem to struggle the most, in particular the ViT_L which does not exceed 10 percent of the baseline. In the non-ID situation, instead no model manages to exceed 5% of initial performance.

For the CNNs the configurations choosen are the following:

- **CNN 0:**
 - 1st conv. layer filters (n1): 128
 - 2nd conv. layer filters (n2): 256
 - Dropout Rate: 0.2
- **CNN 1:**
 - 1st conv. layer filters (n1): 128
 - 2nd conv. layer filters (n2): 512
 - Dropout Rate: 0.2
- **CNN 2:**
 - 1st conv. layer filters (n1): 256
 - 2nd conv. layer filters (n2): 256
 - Dropout Rate: 0.3
- **CNN 3:**
 - 1st conv. layer filters (n1): 64
 - 2nd conv. layer filters (n2): 128
 - 3rd conv. layer filters (n3): 192
 - Dropout Rate: 0.2
- **CNN 4:**
 - 1st conv. layer filters (n1): 64
 - 2nd conv. layer filters (n2): 128
 - 3rd conv. layer filters (n3): 256
 - Dropout Rate: 0.2

C. Privacy

The gradient inversion attack aims to reconstruct the input image or set of images of a neural network from its gradients.

TABLE III
FEDMD TEST ACCURACY RESULT

Model	Baseline accuracy (%)	Final accuracy (%)	Upper bound accuracy (%)
CNN 0	61.8	71	83
CNN 1	54.3	70.7	83.1
CNN 2	56.9	68.2	82.7
CNN 3	66.8	73.3	85.9
CNN 4	60.6	75	86.5
ResNet Group	57.1	67.2	72.1
ResNet Batch	47.1	72.3	83.3
ViT_S	51.2	65.4	68.4
ViT_B	52.6	64.5	71.8
ViT_L	51.4	58.7	67.2

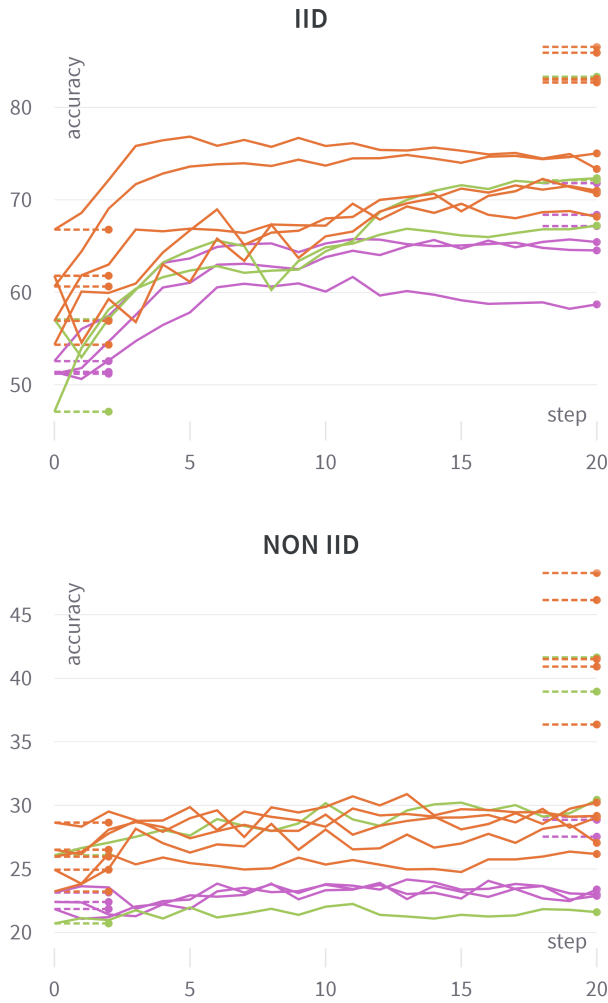


Fig. 3. FedMD results after 20 steps. Orange lines represents the CNNs, green lines represents ResNets and the purple ones represents the Vision Transformer models. The dashed lines on the left indicates the baseline before to start the collaboration phase and the same line on the rights indicates the upper bounds.

As described in [7] this attack is able to trace the input data even for very deep, pre-trained networks, so we tested its performance on our ResNet20 to see how it performs. To accomplish that, we used the pre-trained version done in the centralized training phase, with 160 epochs, moreover we tried to recover both a single image and a batch of images of different size (4, 8, 64), to test the attack in plausible scenarios. We tested different types of environments, but we got the best results, considering all cases, using the cosine similarity-based cost function seen earlier (7) and the Adam optimizer. Regarding the single image in Figure 4 we can see that the reconstruction is quite reliable and certainly recognizable. In this case 8000 iterations were sufficient, with a learning rate of 0.1 and a total variation (TV) of $1e-2$.

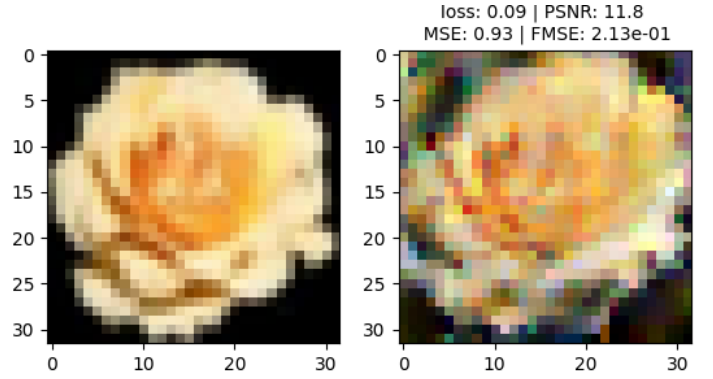


Fig. 4. Single image reconstruction, CIFAR100, 8000 iterations. On the left the ground-truth image, on the right the reconstructed image

As we keep increasing the number of images, we can see that the reconstruction tends to get much worse, however, some features of some images remain recognizable. Concerning the batch of 4 images in Figure 5, we can see that the mushroom of the third image, starting from the left, is faithfully reproduced. A problem that immediately comes to the eye, however, is that this image is shifted with respect to the original and the reason would be attributable to the use of data augmentation in the training phase [7]. In this case, in order to achieve satisfactory results, we had to increase the number of iterations to 24000, the learning rate to 1 and to decrease the total variation to $1e-6$. This configuration will also be used for the versions with batches of 8 and 64 images.

In the batch of 8 images in Figure 6 the reconstruction quality is lower, the method is able to infer some main characteristics of the ground truth images, such as main color and even the general pattern, as can be seen on the first two images from the left to right.

Finally we can see that for the batch of 64 we have obtained pretty satisfactory results, in effect, some images can be mostly recognized. It should be noted that, as explained in section [7] a greater quantity of restarts can enhance the attack's efficacy. Nevertheless, this necessitates additional computational resources.

VI. APPENDIX

Our implementation of the architectures and the algorithms previously mentioned can be found in <https://github.com/AMLProject-PattiPellegrinoRolandi>

A. Multihead self-attention

Standard **qkv** self-attention (SA) is a popular building block for neural architectures. For each element in an input sequence $\mathbf{z} \in \mathbb{R}^{N \times D}$, we compute a weighted sum over all values $\mathbf{v}$ in the sequence. The attention weights A_{ij} are based on the pairwise similarity between two elements of the sequence and their respective query $\mathbf{q}^i$ and key $\mathbf{k}^j$ representations.

$$[\mathbf{q}, \mathbf{k}, \mathbf{v}] = \mathbf{z} \mathbf{U}_{qkv}, \quad \mathbf{U}_{qkv} \in \mathbb{R}^{(D \times 3D_h)} \quad (7)$$

$$A = \text{softmax}(\mathbf{q}\mathbf{k}^T / \sqrt{D_h}), \quad A \in \mathbb{R}^{(N \times N)} \quad (8)$$

$$SA(\mathbf{z}) = A\mathbf{v} \quad (9)$$

Multihead self-attention (MSA) is an extension of SA in which we run k self-attention operations, called “heads”, in parallel, and project their concatenated outputs. To keep compute and number of parameters constant when changing k , D_h (7) is typically set to D/k .

B. ResNet20 Architecture

